Adapter Design Pattern

Study notes - structural pattern, first-principles walkthrough with FAANG interview angle

1. The Problem (First Principles)

Real-life analogy: you travel from India to the US with your laptop charger. The charger works fine, but its plug shape (Indian) does not match the US wall socket shape. The electronics are not broken - the **interface** is incompatible.

You cannot rebuild the US socket (code you do not own - a third-party library) and you do not want to buy a whole new charger (rewrite working code). Instead you buy a small **travel plug adapter** that sits between the two and translates one shape into the other.

In software this happens whenever:

- The **client** code expects a certain interface (e.g. PaymentProcessor.pay(amount)).
- An **existing class** (Adaptee) has the behavior you need, but a different, incompatible interface (e.g. StripeGateway.makeTransaction(amount, currency)).
- You cannot change the client (already built around the interface) and often cannot/should not change the Adaptee (third-party SDK or legacy code).

2. Standard (GoF) Definition

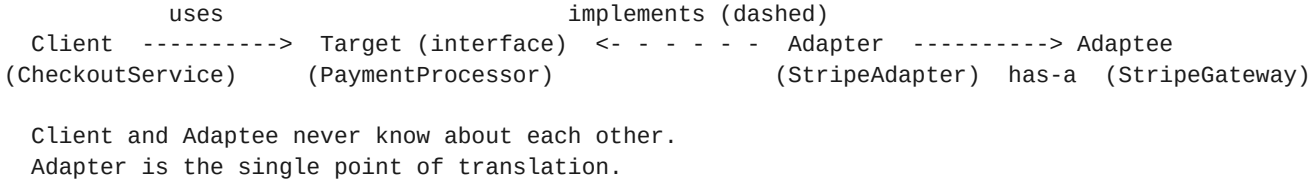
"Convert the interface of a class into another interface the clients expect. Adapter lets classes work together that couldn't otherwise work together because of incompatible interfaces."

It is a **structural** design pattern - it is about how classes/objects are composed, not about behavior or object creation.

3. Structure & Participants

Four roles and three relationships:

- **Client** - your business logic (e.g. CheckoutService). Depends only on the Target interface.
- **Target** - the interface the client already expects (e.g. PaymentProcessor). The 'US wall socket'.
- **Adaptee** - existing class with the needed behavior, wrong interface (e.g. StripeGateway). The 'Indian charger'.
- **Adapter** - implements Target (realization) and holds a reference to Adaptee (composition). Translates every Target call into the right Adaptee call(s).



Text rendering of the UML class diagram: Client -> Target (association), Adapter -> Target (realization/implements, dashed line), Adapter -> Adaptee (composition/has-a).

4. Code Example

The problem - without an Adapter

```
interface PaymentProcessor {
    void pay(double amountInUSD);
}

class StripeGateway {                 // third-party SDK, you can't edit this
    void makeTransaction(double amount, String currency) {
        System.out.println("Charging $" + amount + " " + currency + " via Stripe");
    }
}

class CheckoutService {
    private PaymentProcessor processor;
    public CheckoutService(PaymentProcessor processor) { this.processor = processor; }
    public void checkout(double amount) { processor.pay(amount); }
}
```

CheckoutService only knows how to talk to a PaymentProcessor. StripeGateway does not implement that interface - different method name, different signature. You cannot pass a StripeGateway into CheckoutService. Compile error.

The solution - with an Adapter

```
class StripeAdapter implements PaymentProcessor {
    private StripeGateway stripeGateway;

    public StripeAdapter(StripeGateway stripeGateway) {
        this.stripeGateway = stripeGateway;
    }

    @Override
    public void pay(double amountInUSD) {
        stripeGateway.makeTransaction(amountInUSD, "USD");   // translation happens here
    }
}

// Usage
PaymentProcessor processor = new StripeAdapter(new StripeGateway());
CheckoutService checkout = new CheckoutService(processor);
checkout.checkout(49.99);
// Output: Charging $49.99 USD via Stripe
```

Zero changes to CheckoutService and zero changes to StripeGateway. The adapter is the only new piece.

5. Types of Adapter

Type 1 - Object Adapter (composition-based)

The Adapter holds a reference to an instance of the Adaptee and delegates calls to it (shown above).

Analogy: the physical travel plug adapter. It is a separate object that holds your charger plugged into it - you could unplug your charger and plug in a completely different device. The adapter does not care what is inside it. This is the flexibility composition gives you.

Type 2 - Class Adapter (inheritance-based)

The Adapter inherits from both Target and Adaptee simultaneously - requires multiple inheritance, so practically only viable in C++ (Java/C# only allow single class inheritance).

```
class StripeClassAdapter : public PaymentProcessor, private StripeGateway {
public:
    void pay(double amount) override {
        makeTransaction(amount, "USD");   // inherited method, called directly
    }
};
```

Analogy: instead of buying a separate plug adapter, a factory permanently rebuilds your charger's internals so its plug is welded into a US-style plug, soldered onto the original circuit. A single fused object - fixed at manufacture time, cannot swap what's underneath.

	Object Adapter	Class Adapter
Mechanism	Composition (has-a)	Multiple inheritance (is-a + is-a)
Language support	Any OOP language	Only languages with multiple inheritance (C++)
Adapt subclasses too?	Yes - holds a reference, polymorphic	No - bound to one class at compile time
Flexibility	Adaptee swappable at runtime	Fixed at compile time
Preferred in practice	Yes - matches “favor composition”	Rarely used, mostly academic/C++

Preferred answer in interviews: **Object Adapter** - composition gives runtime flexibility, avoids the fragile-base-class problem, and works even without multiple inheritance.

6. Real-World Use Cases

- **Java I/O:** InputStreamReader adapts a byte-based InputStream (Adaptee) to the character-based Reader interface (Target).
- **Collections.list(Enumeration):** adapts the legacy Enumeration interface to Iterator/List.
- **Payment gateway integration:** unifying Stripe, PayPal, Razorpay behind one PaymentProcessor interface.
- **Third-party/legacy library integration** generally - adopting a new interface standard while old code/vendor SDKs speak a different dialect.
- **XML/JSON parser wrapping:** adapting different serialization libraries behind one Serializer interface.
- **Multi-cloud SDKs:** adapting AWS S3, GCS, Azure Blob behind one FileStorage interface so app logic stays cloud-agnostic.

7. FAANG Interview Angle - Distinguishing Adapter From Its Cousins

- **vs Facade:** Adapter makes an existing interface compatible with what the client expects (one-to-one translation, no new behavior). Facade creates a new, simplified interface over a complex subsystem of many classes (many-to-one simplification).
- **vs Decorator:** both wrap an object, but Decorator keeps the same interface and adds new responsibilities/behavior. Adapter changes the interface entirely so incompatible things can talk.

- **vs Bridge:** structurally similar (composition, delegation) but different intent. Adapter is applied after the fact, retrofitting compatibility between independently designed things (often a legacy/third-party class). Bridge is designed up front, deliberately splitting an abstraction from its implementation so both can evolve independently.
- **Common curveball:** “Does Adapter break the Open/Closed Principle?” Good answer: it upholds OCP - you extend behavior by adding a new Adapter class, never modifying the existing Client or Adaptee.

8. Memory Map (Quick Recall)

```
ADAPTER PATTERN
|
+-- PROBLEM: Client expects interface A, existing class only offers interface B
|     (analogy: US socket vs Indian plug)
|
+-- DEFINITION: Convert one class's interface into another the client expects,
|               so incompatible classes can collaborate
|
+-- 4 PARTICIPANTS
|     +-- Client    -> depends only on Target
|     +-- Target    -> interface client already expects
|     +-- Adaptee   -> existing class, incompatible interface
|     +-- Adapter   -> implements Target + wraps Adaptee -> translates calls
|
+-- 2 TYPES
|     +-- Object Adapter -> composition (has-a)     -> preferred, any language
|     +-- Class Adapter  -> multi-inheritance        -> C++ only, rigid
|
+-- REAL EXAMPLES: InputStreamReader, Enumeration->Iterator, payment gateways
|
+-- INTERVIEW DIFFERENTIATORS
      +-- vs Facade    -> 1:1 translation, not many:1 simplification
      +-- vs Decorator -> changes interface, not just adds behavior
      +-- vs Bridge    -> retrofit after the fact, not designed up front
      +-- upholds OCP  -> extend via new class, never modify existing ones
```

One sentence to remember: Adapter lets two interfaces that were never designed to work together, work together - without touching either one.